1) Write a program to implement single Link List (with function insertion at first node, insertion at last node, deletion at first node, deletion at last node, insertion and deletion at middle node and traversing of link list).

```cpp
#include <bits/stdc++.h>
using namespace std;
struct Node{
    int data;
    Node*next;
    Node(int data){
        this->data=data;
        this->next=nullptr;
    }
};
Node* insertatFront(int data,Node*head){
    Node* newnode=new Node(data);
    newnode->next=head;
    return newnode;
}
Node* insertatBack(int data,Node*head){
    if (head==nullptr){return head;}
    Node*curr=head;
    while(curr->next!=nullptr){
        curr=curr->next;
    }
    Node* newnode=new Node(data);
    curr->next=newnode;
    newnode->next=nullptr;
    return head;
}
void printy(Node* head){
    Node*curr=head;
    while(curr!=nullptr){
        cout << curr->data << "-";
        curr=curr->next;
    }
    cout << "\n";
}
Node*deleteFirstNode(Node* head){
    Node*temp=head;
    head=head->next;
    delete temp;
    return head;
}
```

```
}
Node*deleteFirstNode(Node* head){
    Node*temp=head;
    head=head->next;
    delete temp;
    return head;
}
Node*deleteLastNode(Node* head){
    Node*curr=head;
    while(curr->next->next!=nullptr){
        curr=curr->next;
    }
    curr->next=nullptr;
    return head;
}
Node*insertAtMiddle(Node*head,int data){
    Node*slow=head;
    Node*fast=head;
    while(fast!=nullptr&&fast->next!=nullptr){
        slow=slow->next;
        fast=fast->next->next;
    }
    Node*newnode=new Node(data);
    Node*temp=slow->next;
    slow->next=newnode;
    newnode->next=temp;
    return head;
}
Node* deleteAtMiddle(Node*head){
    Node*slow=head;
    Node*fast=head;
    while(fast!=nullptr&&fast->next!=nullptr){
        slow=slow->next;
        fast=fast->next->next;
    }
    if (slow->next != nullptr) {
    Node* temp = slow->next;
    slow->next = slow->next->next;
    delete temp;
    }
    return head;
}
int main (){
```

```
int main (){
    cout << "please insert data in linked list ";
    int x;
    Node*head=nullptr;
    Node*tail=nullptr;
    while(cin>>x){
        Node* new_node=new Node(x);
        if (head==nullptr){
            head=new_node;
            tail=new_node;
        }
        else {
            tail->next=new_node;
            tail=new_node;
        }
        if (cin.peek()=='\n') break;
    }
    cout << "enter the element to insert at front : ";
    int fe;
    cin>>fe;
    head=insertatFront(fe,head);
    printy(head);
    cout << "enter the element to insert at back : ";
    int be;
    cin>>be;
    head=insertatBack(be,head);
    printy(head);
    cout << "now deleting the first node : ";
    head=deleteFirstNode(head);
    printy(head);
    cout << "now deleting the last node : " ;
    head=deleteLastNode(head);
    printy(head);
    cout << "enter the element to insert at middle node :
    int me;
    cin>>me;
    head=insertAtMiddle(head,me);
    printy(head);
    cout << "delete at middle : ";
    head=deleteAtMiddle(head);
    printy(head);
return 0;
}
```

2) Write a program to implement Doubly Link List (with function insertion at first node, insertion at last node, deletion at first node, deletion at last node, insertion and deletion at middle node and traversing of link list).

```
#include <bits/stdc++.h>
using namespace std;
struct Node{
    int data;
    Node*next=nullptr;
    Node*prev=nullptr;
    Node(int data){
        this->data=data;
        this->next=nullptr;
        this->prev=nullptr;
    }
};
Node* insertAtFirst(Node*head,int data){
    Node* newnode=new Node(data);
    head->prev=newnode;
    newnode->prev=nullptr;
    newnode->next=head;
    return newnode;
}
Node* insertAtBack(Node*head,int data){
    Node*curr=head;
    while(curr->next!=nullptr){
        curr=curr->next;
    }
    Node*newnode=new Node(data);
    curr->next=newnode;
    newnode->next=nullptr;
    newnode->prev=curr;
    return head;
}
Node* deletefirstnode(Node *head){
    Node*temp=head;
    head=head->next;
    head->prev=nullptr;
    delete temp;
    return head;
}
Node* insertAtMiddle(Node*head,int data){
    Node*slow=head;
    Node*fast=head;
    while(fast!=nullptr&&fast->next!=nullptr){
        slow=slow->next;
        fast=fast->next->next;
```

```
        fast=fast->next->next;
    }
    Node* newnode=new Node(data);
    Node*temp=slow->next;
    slow->next=newnode;
    newnode->next=temp;
    temp->prev=newnode;
    newnode->prev=slow;
    return head;
}
Node* deleteatback(Node*head){
    Node*curr=head;
    while(curr->next->next!=nullptr){
        curr=curr->next;
    }
    curr->next=nullptr;
    return head;
}
Node *deleteAtMiddle(Node*head){
    Node*slow=head;
    Node*fast=head;
    while(fast!=nullptr && fast->next!=nullptr){
        slow=slow->next;
        fast=fast->next->next;
    }
    if (slow == head) {
        head = head->next;
        head->prev = nullptr;
    }
    else {
        if (slow->prev) slow->prev->next = slow->next;
        if (slow->next) slow->next->prev = slow->prev;
    }

    delete slow;
    return head;
}
void printy(Node*head){
    Node*curr=head;
    while(curr!=nullptr){
        cout << curr->data << "-" ;
        curr=curr->next;
    }
```

```
34 int main (){
33     Node*head=nullptr;
32     Node*tail=nullptr;
31     int x;
30     while(cin>>x){
29         Node* newnode=new Node(x);
28         if (head==nullptr){
27             head=newnode;
26             tail=newnode;
25         }
24         else {
23             tail->next=newnode;
22             newnode->prev=tail;
21             tail=newnode;
20         }
19         if (cin.peek()=='\n') break;
18     }
17     int fe;
16     cout << "enter the element at first : ";
15     cin >> fe;
14     head=insertAtFirst(head,fe);
13     printy(head);
12     int be;
11     cout << "enter the element at back : ";
10     cin >> be;
 9     head=insertAtBack(head,be);
 8     printy(head);
 7     cout << "delettion of first node : ";
 6     head=deletefirstnode(head);
 5     printy(head);
 4     cout << "deletion at back : ";
 3     head=deleteatback(head);
 2     printy(head);
 1     cout << "insert a element at middle : ";
122     int me;
 1     cin >> me;
 2     head=insertAtMiddle(head,me);
 3     printy(head);
 4     cout << "delete at middle : ";
 5     head=deleteAtMiddle(head);
 6     printy(head);
 7 return 0;
 8 }
```

```
}

int main() {
    Node* head = nullptr;
    cout << "Enter elements for initial list (end with newline): ";
    int x;
    while (cin >> x) {
        head = insertAtLast(head, x);
        if (cin.peek() == '\n') break;
    }
    cout << "Initial circular doubly linked list: ";
    printList(head);

    cout << "Enter element to insert at first: ";
    int f; cin >> f;
    head = insertAtFirst(head, f);
    printList(head);

    cout << "Enter element to insert at last: ";
    int l; cin >> l;
    head = insertAtLast(head, l);
    printList(head);

    cout << "Enter element to insert at middle: ";
    int m; cin >> m;
    head = insertAtMiddle(head, m);
    printList(head);

    cout << "Deleting first...\n";
    head = deleteAtFirst(head);
    printList(head);

    cout << "Deleting last...\n";
    head = deleteAtLast(head);
    printList(head);

    cout << "Deleting middle...\n";
    head = deleteAtMiddle(head);
    printList(head);

    return 0;
}
```

3) Write a program to implement Circular Doubly Link List (with function insertion at first node, insertion at last node, deletion at first node, deletion at last node, insertion and deletion at middle node and traversing of link list).

```
#include <bits/stdc++.h>
using namespace std;
struct Node {
    int data;
    Node* next;
    Node* prev;
    Node(int data) {
        this->data = data;
        this->next = this->prev = nullptr;
    }
};
Node* insertAtFirst(Node* head, int data) {
    Node* newnode = new Node(data);
    if (!head) {
        newnode->next = newnode;
        newnode->prev = newnode;
        return newnode;
    }
    Node* tail = head->prev;
    newnode->next = head;
    newnode->prev = tail;
    tail->next = newnode;
    head->prev = newnode;
    return newnode;
}
Node* insertAtLast(Node* head, int data) {
    Node* newnode = new Node(data);
    if (!head) {
        newnode->next = newnode;
        newnode->prev = newnode;
        return newnode;
    }
    Node* tail = head->prev;
    tail->next = newnode;
    newnode->prev = tail;
    newnode->next = head;
    head->prev = newnode;
    return head;
}
Node* deleteAtFirst(Node* head) {
```

```
Node* deleteAtFirst(Node* head) {
    if (!head) return nullptr;
    if (head->next == head) {
        delete head;
        return nullptr;
    }
    Node* tail = head->prev;
    Node* newhead = head->next;
    tail->next = newhead;
    newhead->prev = tail;
    delete head;
    return newhead;
}

Node* deleteAtLast(Node* head) {
    if (!head) return nullptr;
    if (head->next == head) {
        delete head;
        return nullptr;
    }
    Node* tail = head->prev;
    Node* newtail = tail->prev;
    newtail->next = head;
    head->prev = newtail;
    delete tail;
    return head;
}

Node* insertAtMiddle(Node* head, int data) {
    if (!head) return insertAtFirst(head, data);
    Node* slow = head, *fast = head;
    while (fast->next != head && fast->next->next != head) {
        slow = slow->next;
        fast = fast->next->next;
    }
    Node* newnode = new Node(data);
    Node* nxt = slow->next;
    slow->next = newnode;
    newnode->prev = slow;
    newnode->next = nxt;
    nxt->prev = newnode;
    return head;
}
```

```
}

Node* deleteAtMiddle(Node* head) {
    if (!head) return nullptr;
    if (head->next == head) {
        delete head;
        return nullptr;
    }
    Node* slow = head, *fast = head;
    while (fast->next != head && fast->next->next != head) {
        slow = slow->next;
        fast = fast->next->next;
    }
    if (slow == head) return deleteAtFirst(head);
    Node* prev = slow->prev;
    Node* nxt = slow->next;
    prev->next = nxt;
    nxt->prev = prev;
    delete slow;
    return head;
}

void printList(Node* head) {
    if (!head) {
        cout << "List empty\n";
        return;
    }
    Node* temp = head;
    do {
        cout << temp->data << " ";
        temp = temp->next;
    } while (temp != head);
    cout << "\n";
}

int main() {
    Node* head = nullptr;
    cout << "Enter elements for initial list (end with newline): ";
    int x;
    while (cin >> x) {
        head = insertAtLast(head, x);
        if (cin.peek() == '\n') break;
    }
```